

CS2050 PROJECT PROPOSAL

Aditya Saravanakumar and Lev Krulgyak

1 Scope of the Project

The basic goal of our project is to run particle-based simulations at astrophysical scales in order to demonstrate large-scale cosmological phenomena such as galaxy formation, cosmic expansion, void formation, etc. Simulations of this type are the basis for a vast area of research in computational cosmology, so we must carefully choose our scope and set of physical phenomena to model so that we can spend most of our efforts on the HPC aspect of the project. We will thus arrange our project goals in “tiers” of increasing physical complexity and work our way up as time permits.

1.1 N -Body Gravity Solver

One of the simplest ways to observe astrophysical phenomena is with an N -body gravity simulation. Starting with a set I of N particles, we seed some initial positions $\vec{p}_i$, velocities $\vec{v}_i$, and masses m_i and numerically solve the differential equation

$$\frac{d^2\vec{v}_i}{dt^2} = -G \sum_{j \in I} \frac{m_j(\vec{p}_i - \vec{p}_j)}{\|\vec{p}_i - \vec{p}_j\|^{3/2}}.$$

1.2 Smooth Particle Hydrodynamics Solver

Astrophysical phenomena such as galaxy formation and planetary collisions are characterized by strong fluid dynamics and self-gravity effects. Such phenomena elude Newtonian N -body solvers, and smooth particle hydrodynamics (SPH) solvers are often used to resolve these effects. Here, each field $\vec{f}(\vec{r})$ as a smooth convolution $\vec{f}_s(\vec{r})$ with a kernel $W(\vec{r}, h)$ of characteristic length scale h which is then approximated using the particles as:

$$\vec{f}_s(\vec{r}) = \int \vec{f}(\vec{r}') W(\vec{r} - \vec{r}', h) d\vec{r}' \approx \sum_{j \in I} \frac{m_j}{\rho_j} \vec{f}_j W(\vec{r} - \vec{r}_j, h)$$

2 Implementation

Our goal is to be able to run and validate particle simulations at scales up to $N \approx 10^8$. If we assume that every particle takes up 64 bytes, the data of the set of particles would take up about 6.4 Gb.

2.1 Tree Methods

In a naive implementation of an N -body gravity simulation, we are highly compute bound since the memory complexity is $O(N)$ but the computational complexity is $O(N^2)$. A naive implementation has the advantage of being “stupidly parallelizable” so we might start with this implementation.

However, there is plenty of room in memory for acceleration structures which help speed up computation. For instance, the Barnes-Hut algorithm partitions space into an octree and treats distant clusters of particles as larger single masses. With this approach, the computational complexity can be improved to $O(N \log N)$, but parallelizing is trickier since we need to carefully build a tree using multiple threads without race conditions.

2.2 Time Stepping Strategies

Another approach to tackle this computational challenge that we intend to explore is local time-stepping schemes wherein particles are evolved using timestep sizes that depend on local conditions. While this can prove efficient if implemented correctly, it comes with load balancing and synchronization issues.